SUNWAY UNIVERSITY

Jeffrey Cheah Foundation
Nurturing the Seeds of Wisdom

Practical 03:
Java Socket Programming

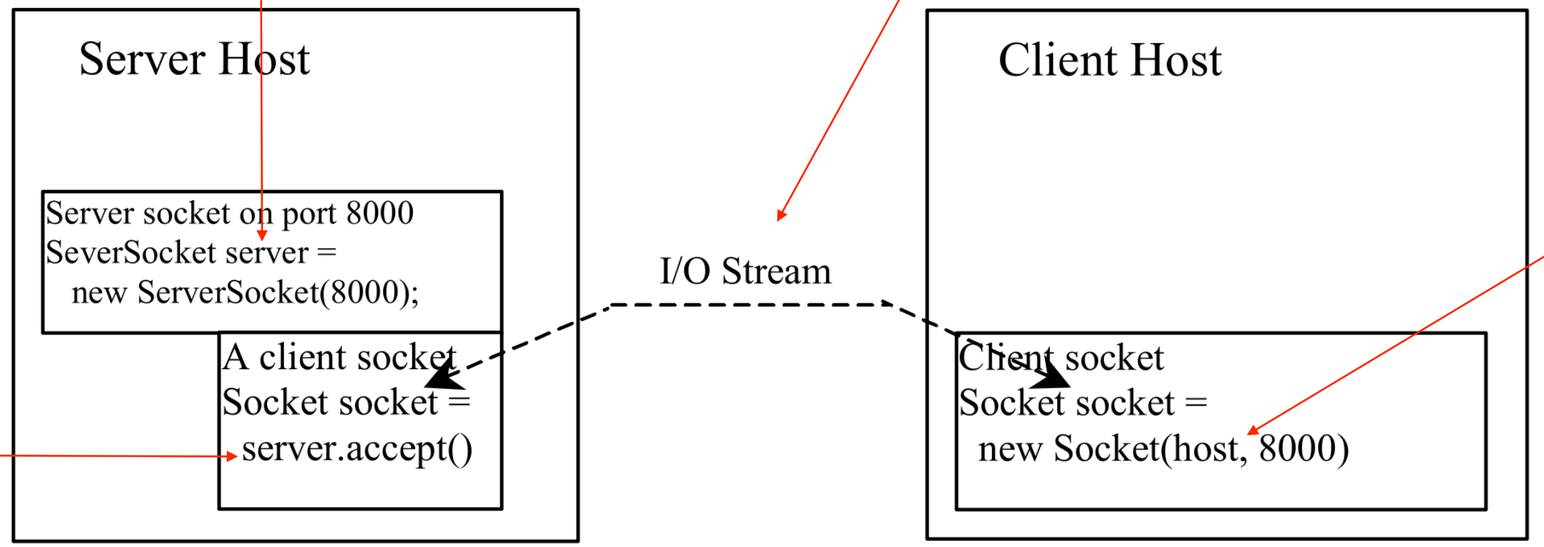
OUR VISION: TO BE A WORLD CLASS UNIVERSI

NET3204
Distributed Systems

Chee Sun LIEW cheesunl@sunway.edu.my

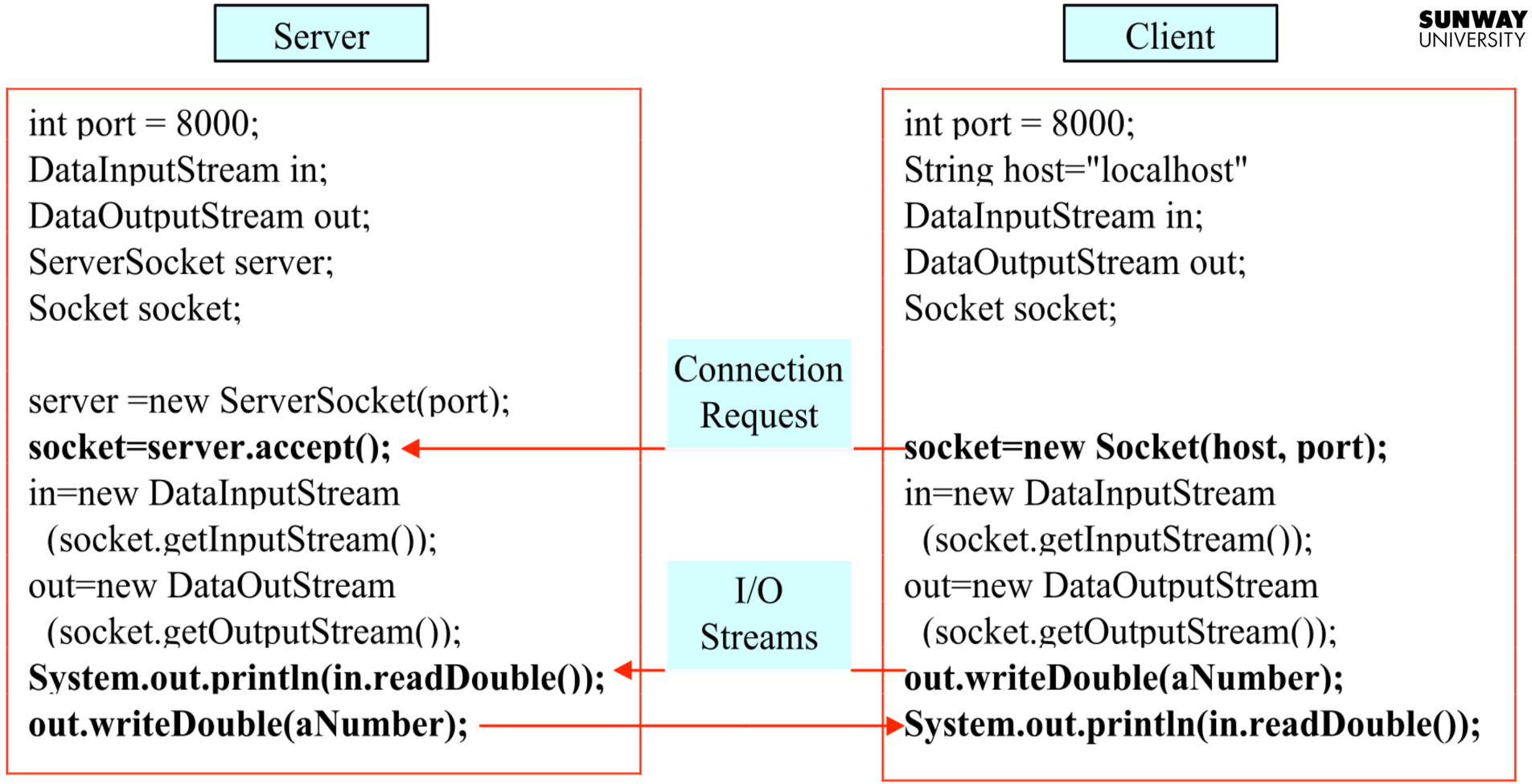
The server must be running when a client starts. The server waits for a connection request from a client. To establish a server, you need to create a server socket and attach it to a port, which is where the server listens for connections.

After the server accepts the connection, communication between server and client is conducted the same as for I/O streams.



After a server socket is created, the server can use this statement to listen for connections.

The client issues this statement to request a connection to a server.




```
InputStream input = socket.getInputStream();
OutputStream output = socket.getOutputStream();
```


OUR VISION: TO BE A WORLD CLASS UNIVERSITY

Sockets used for Streams

- Listening process (server) create a stream socket and binds to the server's socket address and wait for request
- Server accepts connection and obtain a new socket for communication with client

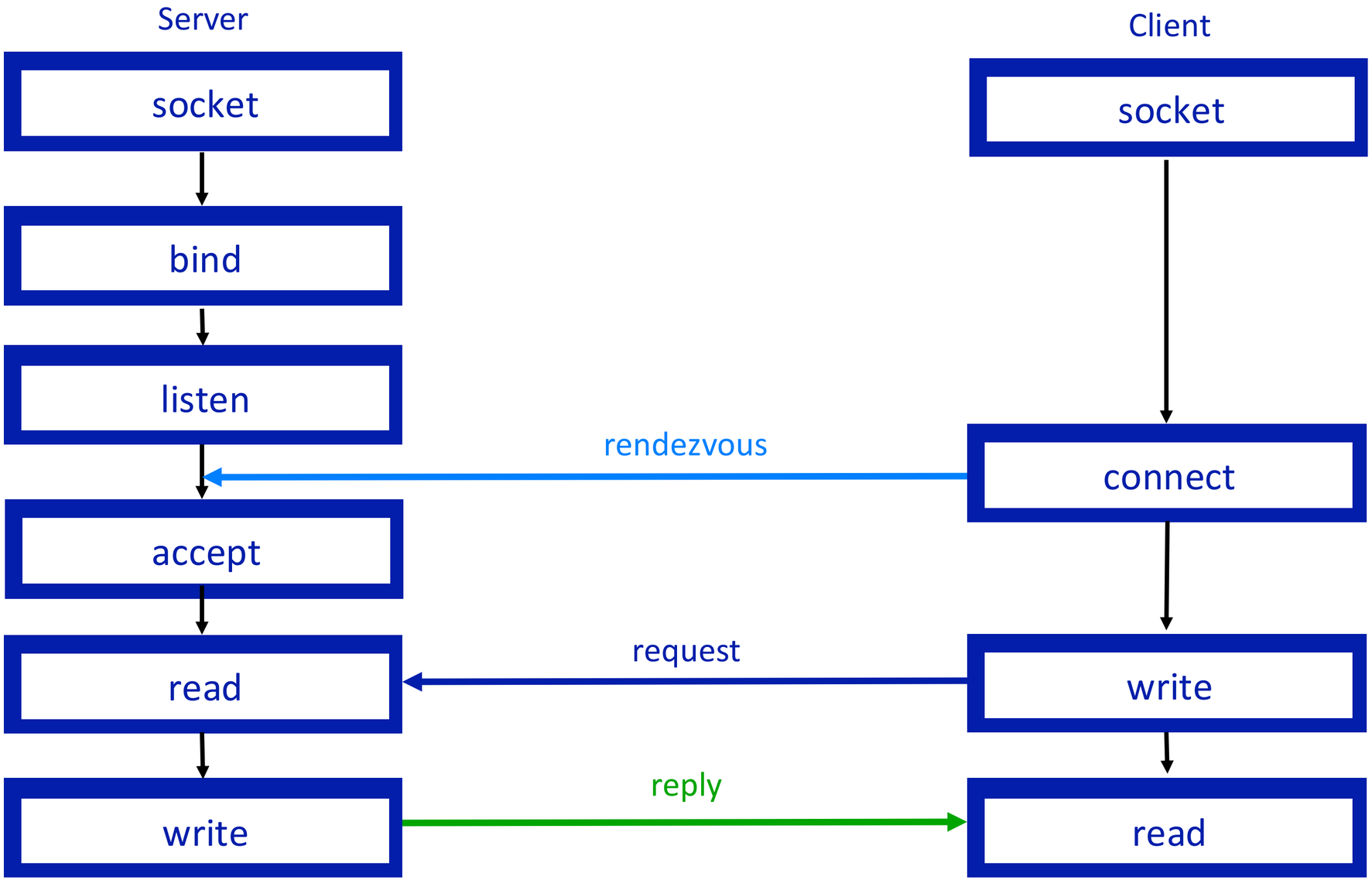
Requesting a connection

```
s = socket(AF_INET, SOCK_STREAM,0)
●
●
connect(s, ServerAddress)
●
●
write(s, "message", length)
```

Listening and accepting a connection

```
s = socket(AF_INET, SOCK_STREAM,0)
●
bind(s, ServerAddress);
listen(s,5);
●
sNew = accept(s, ClientAddress);
●
n = read(sNew, buffer, amount)
```

TCP Client-Server Socket Communication



Socket Used for Datagram

- Both processes create a socket and get a descriptor
- Bind their sockets to socket addresses then communicate

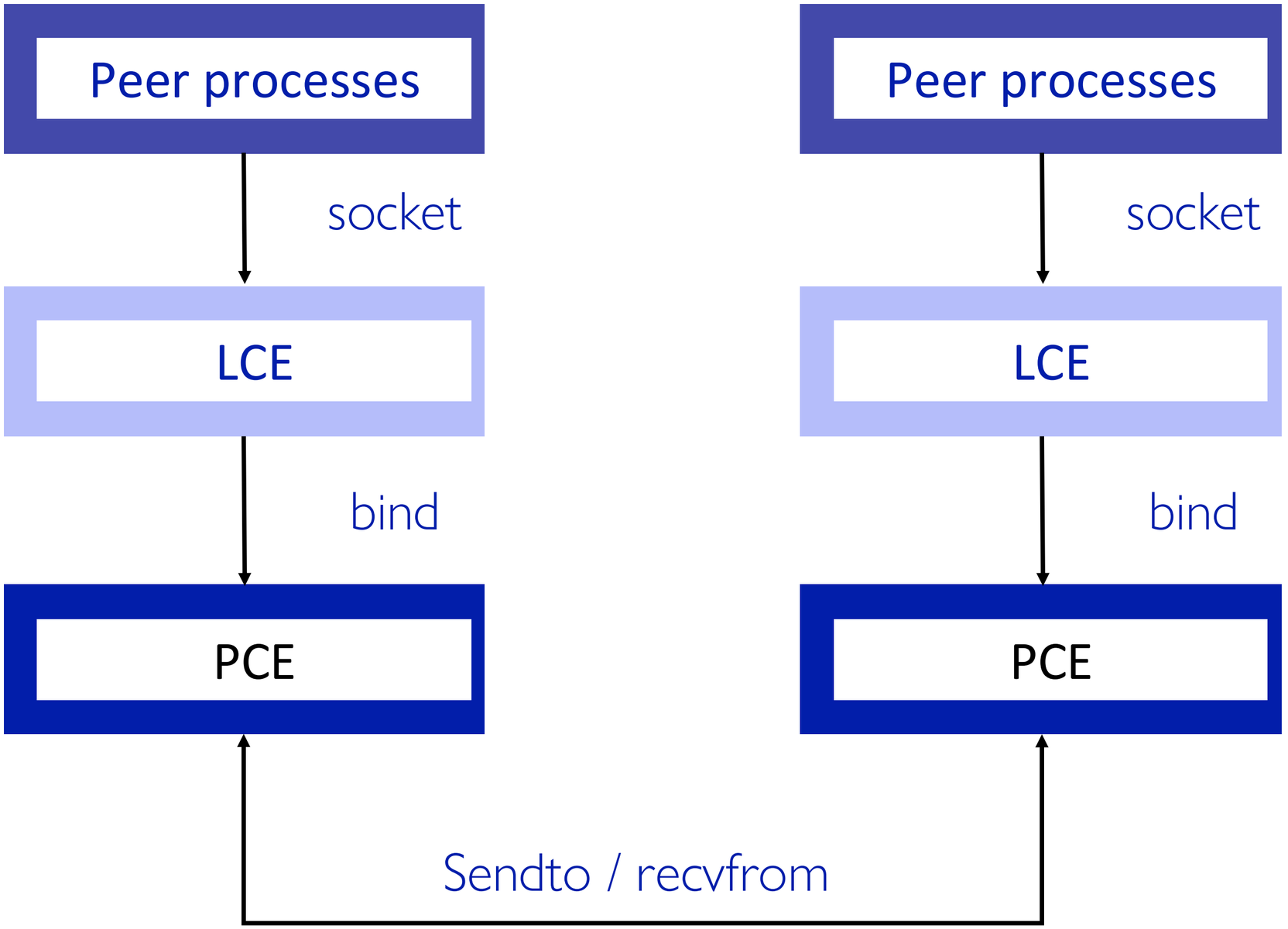
Sending a message

```
s = socket(AF_INET, SOCK_DGRAM, 0)
•
•
bind(s, ClientAddress)
•
•
sendto(s, "message", ServerAddress)
```

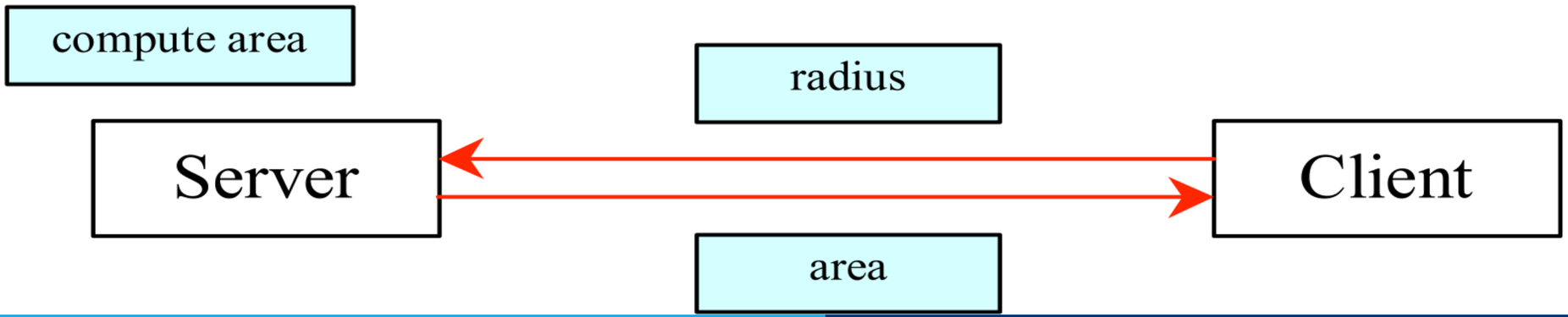
Receiving a message

```
s = socket(AF_INET, SOCK_DGRAM, 0)
•
•
bind(s, ServerAddress)
•
•
amount = recvfrom(s, buffer, from)
```

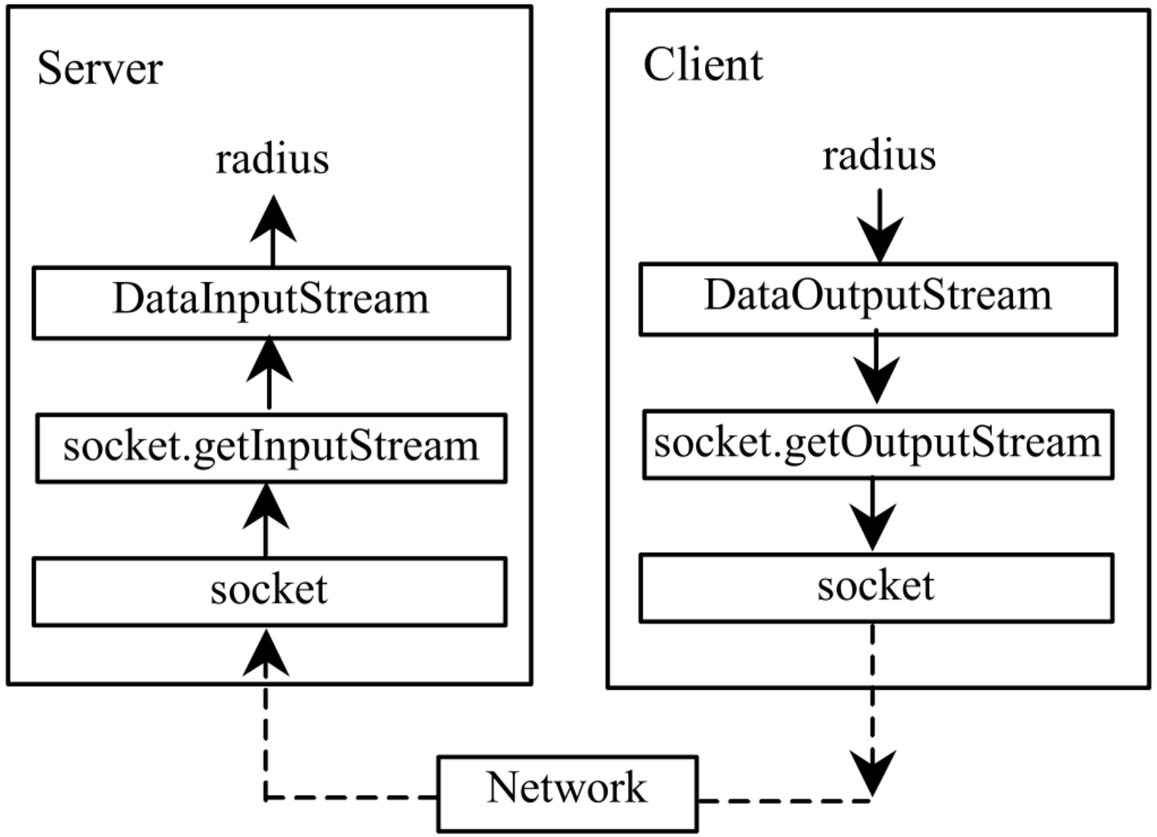
UDP Socket Communication



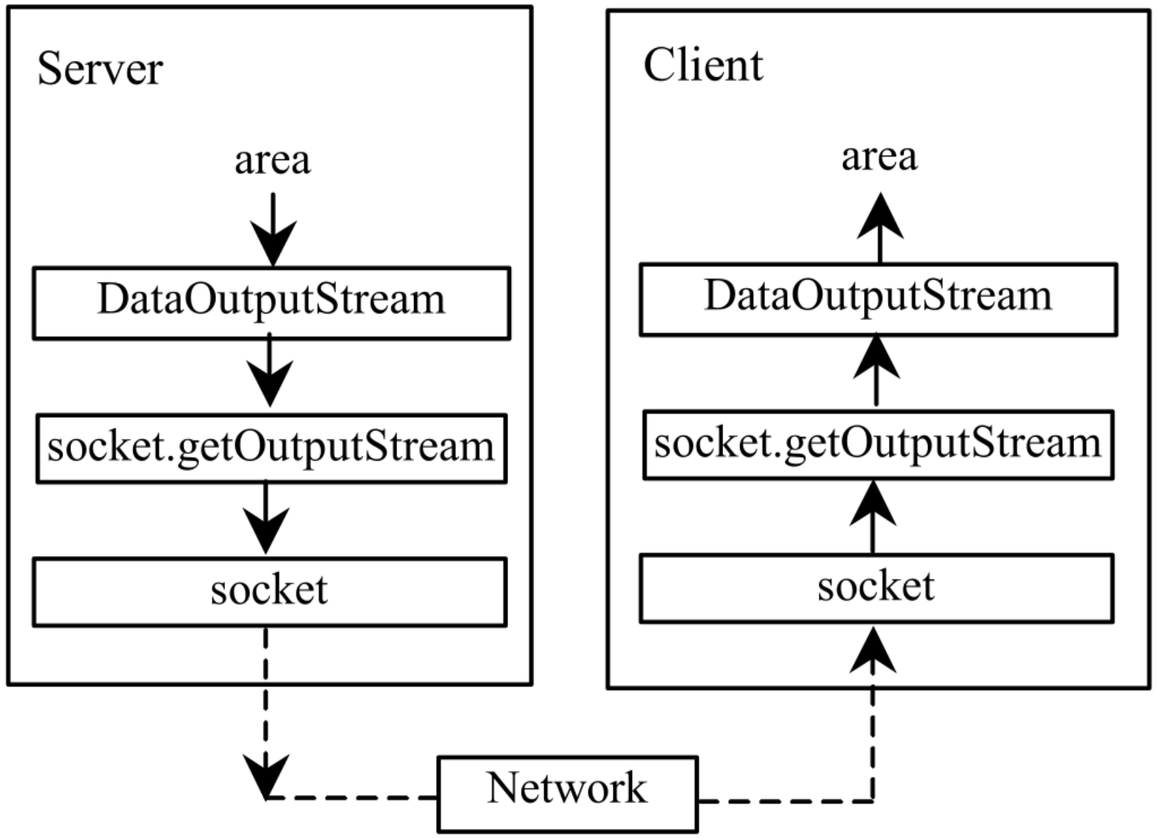
- Problem: Write a client to send data to a server. The server receives the data, uses it to produce a result, and then sends the result back to the client. The client displays the result on the console. In this example, the data sent from the client is the radius of a circle, and the result produced by
the server is the area of the circle.



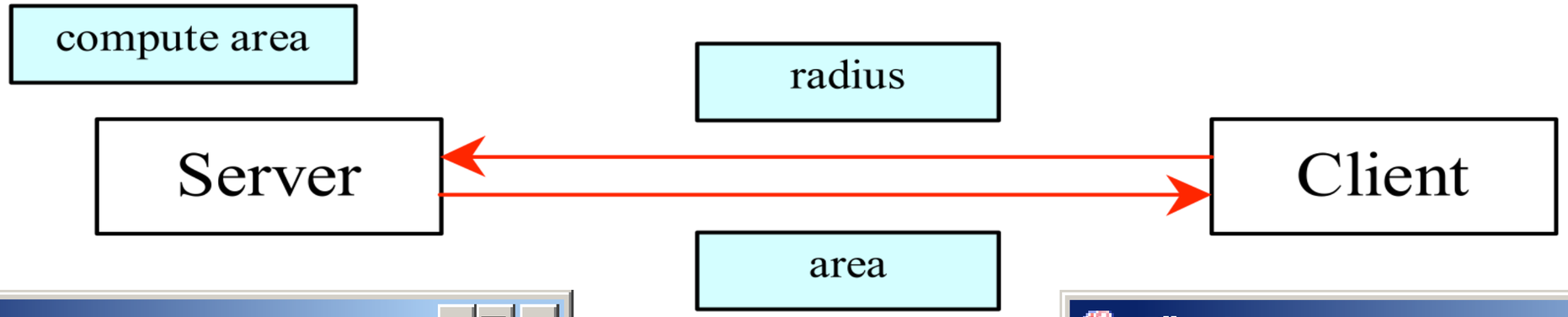
OUR VISION: TO BE A WORLD CLASS UNIVERSITY

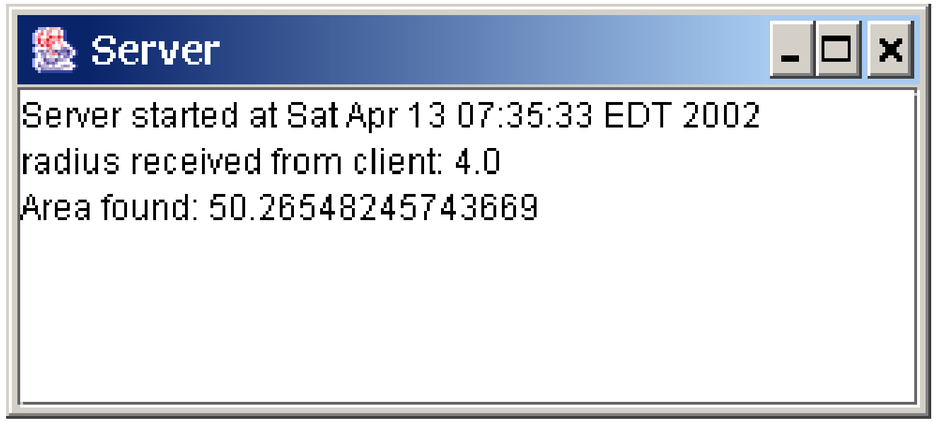

Server
radius
DataInputStream
socket.getInputStream
socket
Client
radius
DataOutputStream
socket.getOutputStream
socket
Network

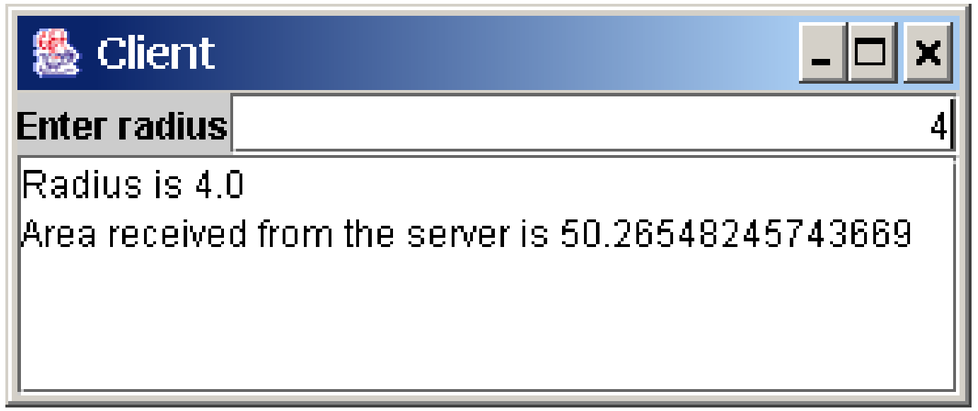
(A)


Server
area
DataOutputStream
socket.getOutputStream
socket
Client
area
DataOutputStream
socket.getOutputStream
socket
Network

(B)


compute area
radius
Server
Client
area


Server
Server started at Sat Apr 13 07:35:33 EDT 2002
radius received from client: 4.0
Area found: 50.26548245743669


Client
Enter radius
4
Radius is 4.0
Area received from the server is 50.26548245743669

Occasionally, you would like to know who is connecting to the server. You can use the InetAddress class to find the client's host name and IP address. The InetAddress class models an IP address. You can use the statement shown below to create an instance of InetAddress for the client on a socket.

```
InetAddress inetAddress = socket.getInetAddress();
```

Next, you can display the client's host name and IP address, as follows:

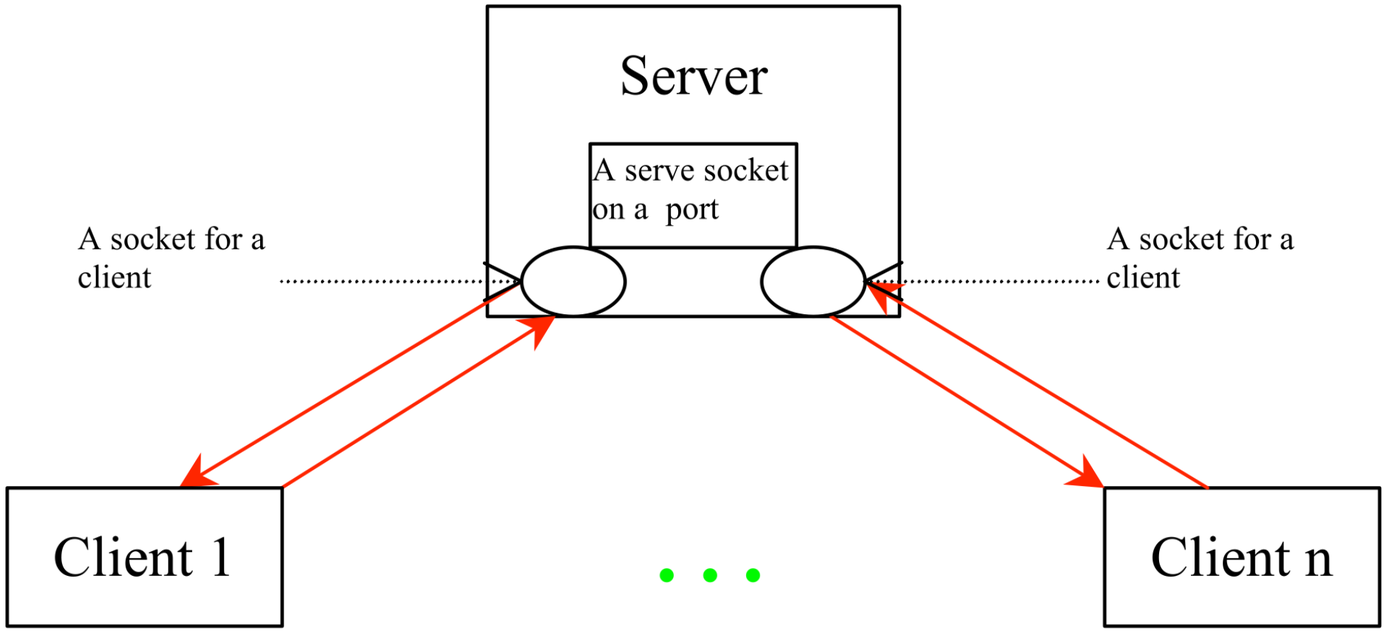
```
System.out.println("Client's host name is " +
  inetAddress.getHostName());
System.out.println("Client's IP Address is " +
  inetAddress.getHostAddress());
```

Multiple clients are quite often connected to a single server at the same time. Typically, a server runs constantly on a server computer, and clients from all over the Internet may want to connect to it. You can use threads to handle the server's multiple clients simultaneously. Simply create a thread for each connection. Here is how the server handles the establishment of a connection:

```
while (true) {
  Socket socket = serverSocket.accept();
  Thread thread = new ThreadClass(socket);
  thread.start();
}
```

The server socket can have many connections. Each iteration of the while loop creates a new connection. Whenever a connection is established, a new thread is created to handle communication between the server and the new client; and this allows multiple connections to run at the same time.

Serving Multiple Clients



Note: Start the server first, then start multiple clients.

Stream socket

- A dedicated point-to-point channel between a client and server.
- Use TCP (Transmission Control Protocol) for data transmission.
- Lossless and reliable.
- Sent and received in the same order.

Datagram socket

- No dedicated point-to-point channel between a client and server.
- Use UDP (User Datagram Protocol) for data transmission.
- May lose data and not 100% reliable.
- Data may not received in the same order as sent.

The DatagramPacket class represents a datagram packet. Datagram packets are used to implement a connectionless packet delivery service. Each message is routed from one machine to another based solely on information contained within the packet.

java.net.DatagramPacket	
length: int	A JavaBeans property to specify the length of buffer.
address: InetAddress	A JavaBeans property to specify the address of the machine where the package is sent or received.
port: int	A JavaBeans property to specify the port of the machine where the package is sent or received.
+DatagramPacket(buf: byte[], length: int, host: InetAddress, port: int)	Constructs a datagram packet in a byte array buf of the specified length with the host and the port for which the packet is sent. This constructor is often used to construct a packet for delivery from a client.
+DatagramPacket(buf: byte[], length: int)	Constructs a datagram packet in a byte array buf of the specified length.
+getData(): byte[]	Returns the data from the package.
+setData(buf: byte[]): void	Sets the data in the package.

DatagramSocket

The DatagramSocket class represents a socket for sending and receiving datagram packets. A datagram socket is the sending or receiving point for a packet delivery service. Each packet sent or received on a datagram socket is individually addressed and routed. Multiple packets sent from one machine to another may be routed differently, and may arrive in any order.

Create a server DatagramSocket

To create a server DatagramSocket, use the constructor DatagramSocket(int port), which binds the socket with the specified port on the local host machine.

Create a client DatagramSocket

To create a client DatagramSocket, use the constructor DatagramSocket(), which binds the socket with any available port on the local host machine.

Sending

To send data, you need to create a packet, fill in the contents, specify the Internet address and port number for the receiver, and invoke the send(packet) method on a DatagramSocket.

Receiving

To receive data, create an empty packet and invoke the receive(packet) method on a DatagramSocket.

Datagram programming is different from stream socket programming in the sense that there is no concept of a ServerSocket for datagrams. Both client and server use DatagramSocket to send and receive packets.

Designate one a server

DatagramServer

```
DatagramSocket socket;
socket = new DatagramSocket(8000);

byte[] buf = new byte[256];

DatagramPacket receivePacket = new
    DatagramPacket(buf, bef.length)

socket.receive(receivePacket);

get data from buf or receivePacket.getData();

DatagramPacket sendPacket = new
   DatagramPacket(buf, bef.length)

fill in the contents in buf;
socket.send(sendPacket);
```

DatagramClient

```
DatagramSocket socket;
socket = new DatagramSocket();

byte[] buf = new byte[256];

InetAddress address = new
    InetAddress(serverName);

DatagramPacket sendPacket = new
    DatagramPacket(buf,  bef.length, address, 8000)

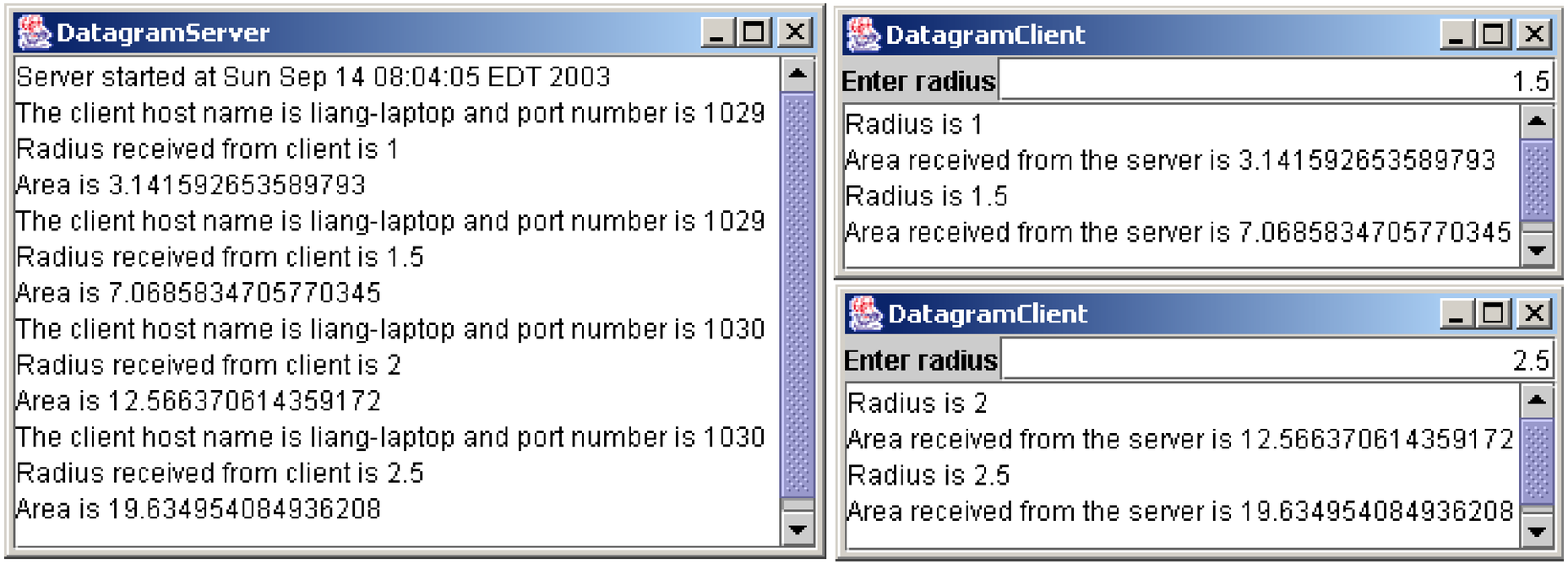
fill in the contents in buf;
socket.send(sendPacket);

DatagramPacket receivePacket = new
    DatagramPacket(buf,  bef.length)

socket.receive(receivePacket);

get data from buf or receivePacket.getData();
```

Example 28.1 presents a client program and a server program using socket streams. The client sends radius to a server. The server receives the data, uses them to find the area, and then sends the area to the client. Rewrite the program using datagram sockets.



OUR VISION: TO BE A WORLD CLASS UNIVERSITY

THANK YOU

Chee Sun LIEW
cheesunl@sunway.edu.my

